ISTA 130

More `return` practice, Traversing Strings with `for` Loops, and Boolean Expressions

Your code must compile.

If your code won't load anything without generating syntax errors, you will get a zero.

If this happens, contact me about the possibility of resubmitting. There will be a small point penalty.

Comment out code causing problems if you run out of time.

Make sure your code still compiles after adding comments!

Do a final check right before submission.

Debugging

Use `print` statements to locate the source of the problem.

If you are really stuck, don't go crazy, send us an email or check in with office hours.

Last time

Python potpourri

- ➔ Variables with augmented assignment.
- ➔ Math with integer division and modulus.
- ➔ The `input` function and its return value type (`str`).
- ➔ Using `for` loops to accumulate values into a variable.
- ➔ Using `return` statements to get information back from function calls.
- ➔ We learned how to index a string to grab copies of individual characters from inside the sequence.

The `input()` function

Takes zero or one argument (unlike `print`) so don't try to separate substrings with commas. Use concatenation instead.

The argument can be a string literal, a string variable, or a string expression.

Returns a string, so you must *typecast* if you need a number (`int()` or `float()`)

Watch your function names.

String Concatenation

All components must be string:

➔ `'Rocket is ' + 1 + ' great dog!'` won't work.
➔ `'Rocket is ' + str(1) + ' great dog!'` will work.

All numbers must be cast to strings when using string concatenation.

When else do we need to use type conversion functions, i.e. cast one type for another?

➔ When we get numbers from a user with the `input` function.

Python's built-in functions

`print` and `input` are built-in functions.

https://docs.python.org/3/library/functions.html

Python returning

Code units must be able to pass information back and forth. Calling code passes information ***to*** functions with arguments.

Functions pass information ***back*** with `return` statements.

GPA example.

Printing vs Returning

Since printing and returning sometimes look similar in the interactive shell (due to the REPL behavior) there is a possible source of confusion.

`print` and `return` do NOT do the same thing!

- ➔ `return` allows your program to communicate internally with itself.
  - The functions in your code call each other passing values back and forth
- ➔ `print` allows your program to communicate externally with humans
  - `print` just puts text on the screen.

Well-structured Code

Now we can start writing longer programs that are logically organized and easier to test, maintain, and understand. We do this by delegating small, logically-related subtasks to functions.

`main` should be short and concise. It should read like an outline of your program.

Traversing a string using a `for` loop

We can combine string indexing with a `for` loop to access every character of a string in order.

This is called ***traversing by index*** and is a hugely important technique.

Conveniently, positions in a string start at 0 and increment. So do the values that the loop control variable refers to. Therefore, we can use the loop control variable (specifically named `i` in the use case) as an index into the string.

Index vs Element

The variable `i` refers to a position inside the string.

The element `string[i]` refers to the character at that position.

Statement: One line of code

A **statement** is the smallest executable unit of code (kind of like a sentence).

For example:

- ➔ tallulah.forward(100)
- ➔ square(tallulah, 50)
- ➔ x = math.sqrt(100)

Expressions: Anything that evaluates to/ returns a value

An **expression** is a combination of literals, variables, operators or functions, which evaluates to a single value.

➔ `2 + 2`

➔ `73.0 * (9 + x)`

➔ `600 + math.sqrt(some_number * 2.0)`

Expressions and Types we have met

- ➔ Arithmetic expressions are combinations of numerical values and arithmetic operators.
  - ◆ For example, `x + y` evaluates to the sum of `x` and `y`
- ➔ Functions evaluate to their `return` values (which might be the `None` object).
- ➔ String expressions (concatenation and repetition) return new strings.
- ➔ We didn't mention this, but most of our turtle methods evaluate to `None`, i.e., `return None`.

Meet the new type `bool`

The `bool` type, Python for Boolean (after mathematician George Boole, inventor of Boolean algebra) has only two values: `True` and `False`.

A Boolean expression is any expression that evaluates to either `True` or `False`.

Today we will meet Boolean expressions involving the comparison operators, including membership.

Comparison Operators

`==`, `!=`, `<`, `>`, `<=`, `>=`

The membership operator: `in`

- ➔ Syntax: `item in collection`
  - Returns `True` if `collection` contains `item`, `False` if not.
- ➔ The only collection type we know so far is `str`, so at this point we can use `in` to determine if a string is a substring of another string (if a character or sequence of characters appear in a string).

The comparison operators all have equal precedence to each other and lower precedence than the arithmetic operators.

Exercise: `bluebird_count`



Practice:

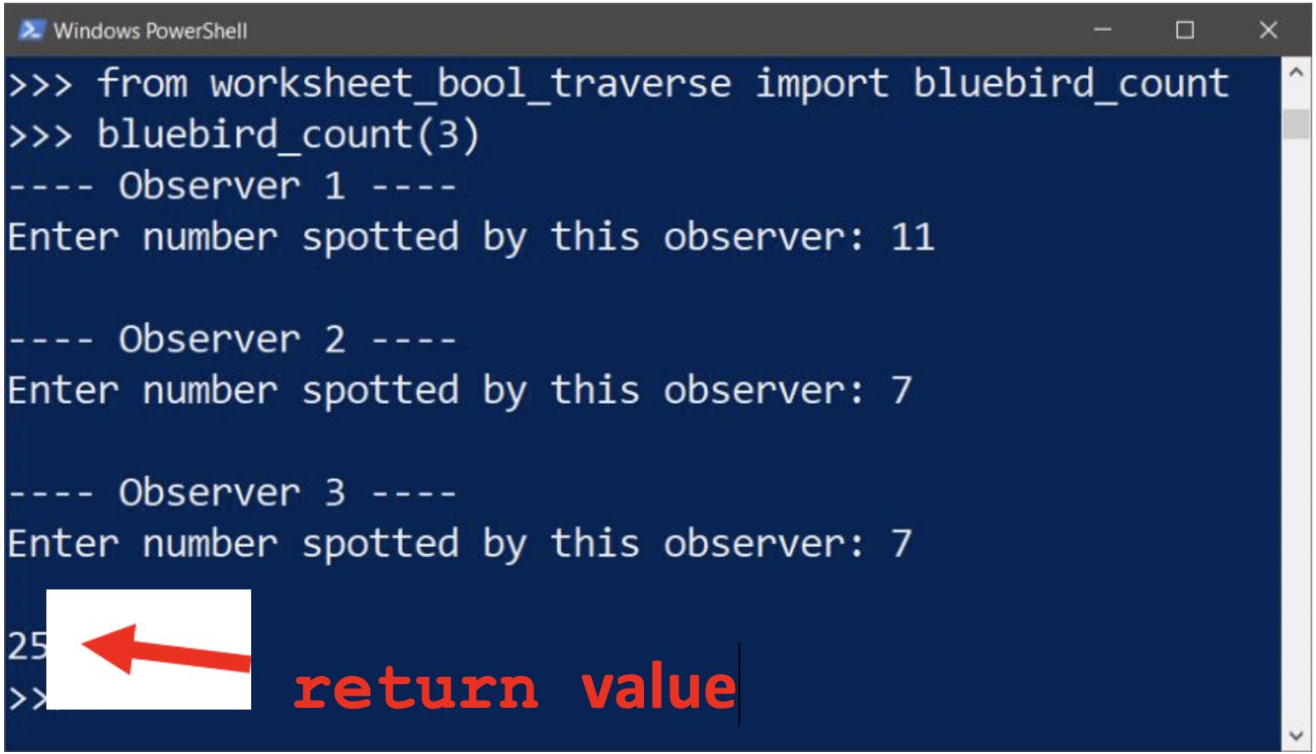
- ➔ Accumulating values into a variable using a loop:
  - ◆ Initialize the variable before the loop
  - ◆ Update the variable in the body of the loop
- ➔ The `input` function.
- ➔ Returning a value from a function.

Exercise: `bluebird_count`



Bird watchers assist biologists by going out together in groups and counting the number of birds they see in a study area. Write a function called **`bluebird_count`** that takes one integer argument representing the number of bird watchers. The function asks the user for the number of birds each observer has spotted. When the data from all observers has been accumulated, the function returns the total number of bluebirds spotted. The next slide shows an example function call.

Exercise: `bluebird_count` – example



Exercise

Write a function called `bluebirds_all_teams` that takes no arguments. Get the number of teams from the user. For each team, get the number of observers from the user and use `bluebird_count` to get all the total number of bluebirds for that team. Accumulate all of the team totals into a regional total and return this value.

Exercise

Write a function called `n_str` that takes a string argument and an integer `n`, and returns a new string that has all of the same characters as the argument, except each one is repeated `n` times.

For example:

```
n_str('dog', 4) → 'ddddoooogggg'
```